310406

310406

B.TECH. III SEM (OLD SCHEME) BACK EXAMINATION 2023-24

COMPUTER SCIENCE AND ENGINEERING

(3CS4–06) - Object Oriented Programming

AD, CSE(AIML), CSDS, CS [3AD4-06, 3AIML4-07, 3CSDS4-07, 3CS4-06]

Time : 3 Hours] *[Max. Marks : 120*

[Min. Passing Marks :

Instructions to Candidates :

Part–A : Short Answer Type Questions (up to **25** words) 10 × 2 = 20 marks. All 10 questions are compulsory.

Part–B : Analytical/Problem Solving questions 5 × 8 = 40 marks. Candidates have to answer 5 questions out of 7.

Part–C : Descriptive/Analytical/Problem Solving questions 4 × 15 marks = 60 marks. Candidates have to answer 4 questions out of 5.

Schematic diagrams must be shown wherever necessary. Any data you feel missing may suitably be assumed and stated clearly. Units of quantities used/calculated must be stated clearly.

Use of the following supporting materials is permitted during examination. (Mentioned in form no. 205).

1_____Nil_____ **2_____Nil_____**

P.T.O.

https://www.btubikaner.com

Part–A

2×10=20

1. What is the main principle behind imperative programming ? [2]
2. Define encapsulation in the context of OOP. [2]
3. Provide an example of abstraction in programming. [2]
4. What is the significance of the "protected" access specifier in inheritance ? [2]
5. What is the purpose of constructors in C++ classes ? [2]
6. What advantage does a friend function offer over a member function ? [2]
7. What is the primary purpose of inheritance in object-oriented programming ? [2]
8. What is the primary purpose of exception handling in C++ ? [2]
9. What is the purpose of a static member function in a C++ class ? [2]
10. What is the keyword used to declare a virtual function in C++ ? [2]

Part–B

8×5=40

1. How do you access a private member variable outside of its class in C++ ? [8]
2. What is an abstract class, and what distinguishes it from a concrete class ? [8]
3. Describe a scenario where default arguments in functions can be useful in C++. [8]
4. Can a friend function access private members of a class directly without an object instance ? Why or why not ? [8]
5. How does an abstract class enforce common behavior among its derived classes ? [8]
6. How does dynamic binding enable runtime polymorphism in C++ ? [8]
7. Explain the relationship between dynamic binding and virtual functions in C++. [8]

https://www.btubikaner.com

Part-C

4×15=60

1. Describe the process of resolving the diamond problem in C++ with virtual inheritance. Explain how virtual base classes help eliminate ambiguity and prevent multiple copies of shared base class components in the inheritance hierarchy. Provide an example to illustrate the use of virtual inheritance in a practical scenario. [15]

2. Given a scenario where you need to design a system for managing a library, propose a class hierarchy using object-oriented principles such as encapsulation, inheritance, and polymorphism. Consider the different entities involved (e.g., books, library members) and their relationships. [15]

3. Describe a scenario where constructors and destructors are essential in managing resources effectively in C++. Provide an example of a class that requires dynamic memory allocation in its constructor and needs to release that memory in its destructor. Explain how constructors and destructors ensure proper resource management in such situations. [15]

https://www.btubikaner.com

4. Describe a scenario where templates would be beneficial in a C++ application. Explain how templates allow for code reusability and flexibility by enabling generic programming. Discuss specific examples of template classes or functions that could be used to solve problems efficiently in the given scenario. [15]

5. Analyze the role of abstract classes and pure virtual functions in facilitating polymorphism in C++. Discuss how abstract classes serve as interfaces for derived classes to implement specific behaviors, promoting code modularity and flexibility. Provide insights into how pure virtual functions enforce common behavior among derived classes while allowing for customization through function overriding. Use examples to illustrate the practical application of these concepts in real-world scenarios. [15]

****

https://www.btubikaner.com